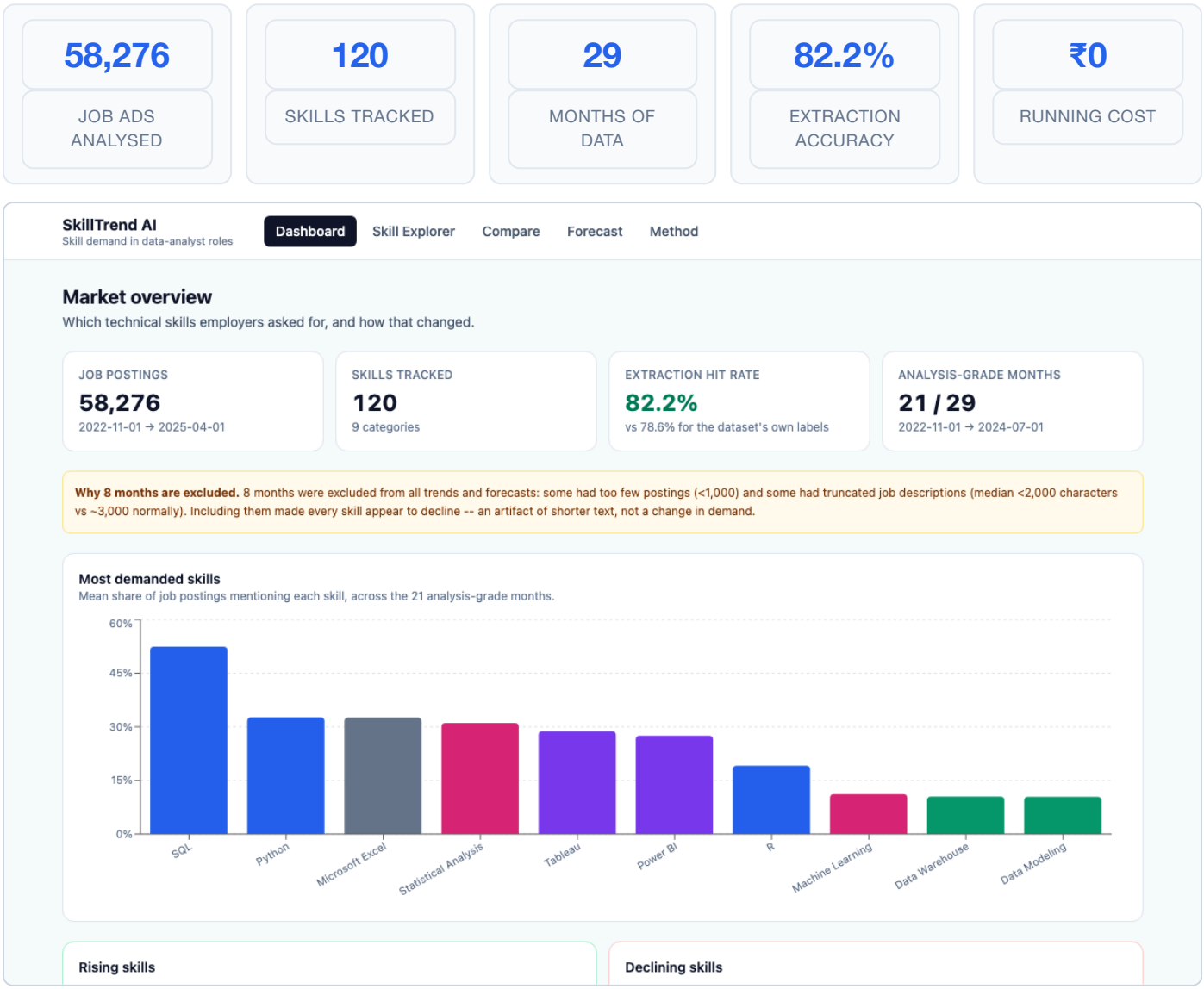
SkillTrend AI

A complete guide to what it is, what it does, and how it was built

SkillTrend AI reads **58,276 real job advertisements** and answers four questions about the skills employers ask for:

1. **What is in demand?** Which technical skills appear most often
2. **How is it changing?** Month-by-month demand over 29 months
3. **What is rising or falling?** Which skills grew or shrank, and by how much
4. **What happens next?** A six-month forecast — with an honest score for how much to trust it



The dashboard. Everything on this page is computed from raw job-ad text by the pipeline described in this document.

IN ONE SENTENCE

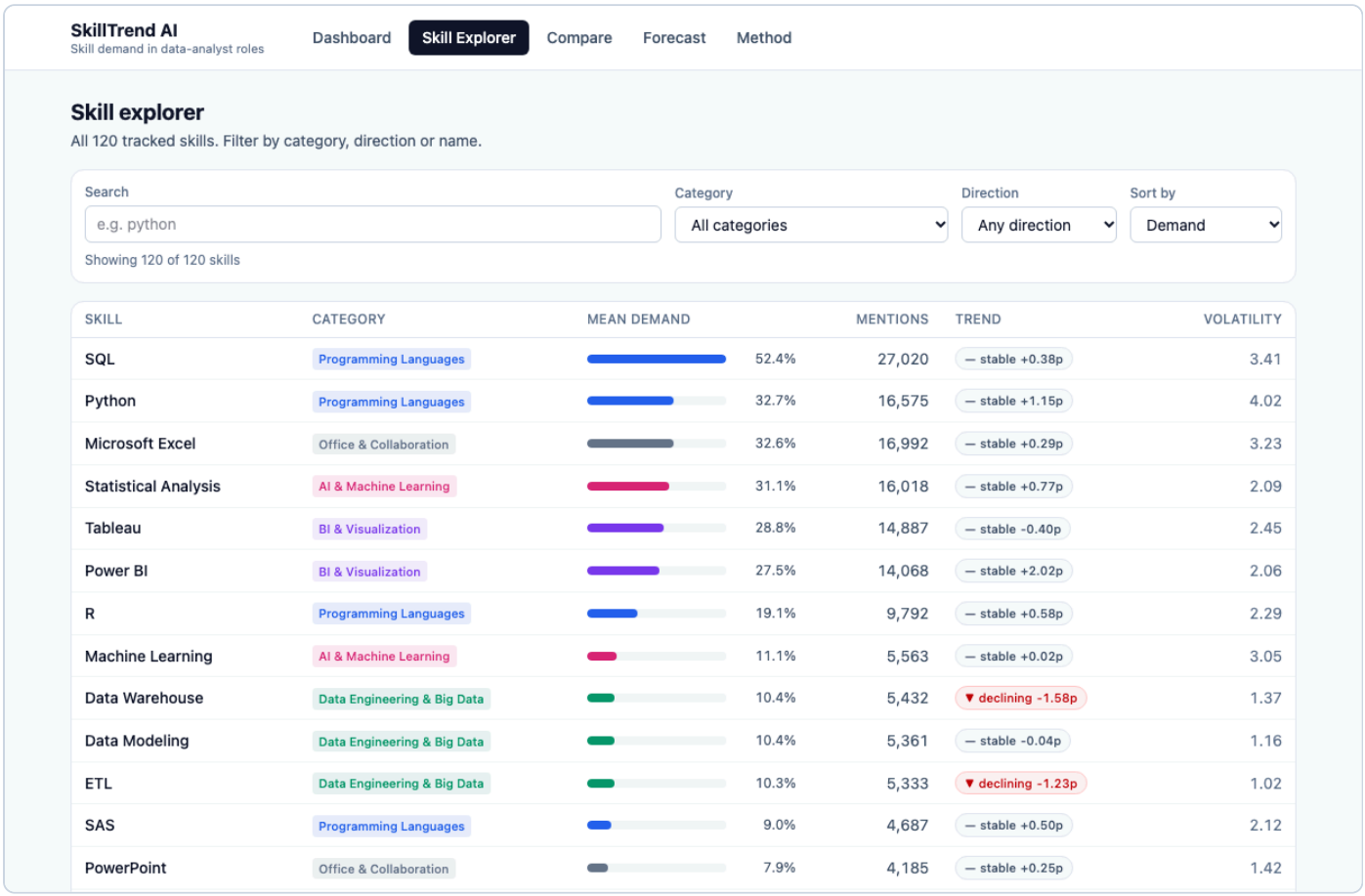
It is a **data pipeline** that turns messy job-ad text into reliable numbers, a **forecasting model** that projects those numbers forward, a **web API** that serves them, and a **dashboard** that makes them explorable — all running on one laptop, with no paid services.

What the application does

Five pages, each answering a different question.

PAGE	WHAT YOU CAN DO
Dashboard	See the headline picture — top skills, what is rising and falling, and how trustworthy the data is
Skill Explorer	Browse all 120 skills; filter by category, by direction, or search by name
Compare	Put up to 6 skills on one chart — "Power BI or Tableau?", "Python or R?"
Forecast	See one skill's history and its 6-month projection, plus how accurate the model actually is
Method	Read how it works and where it falls short

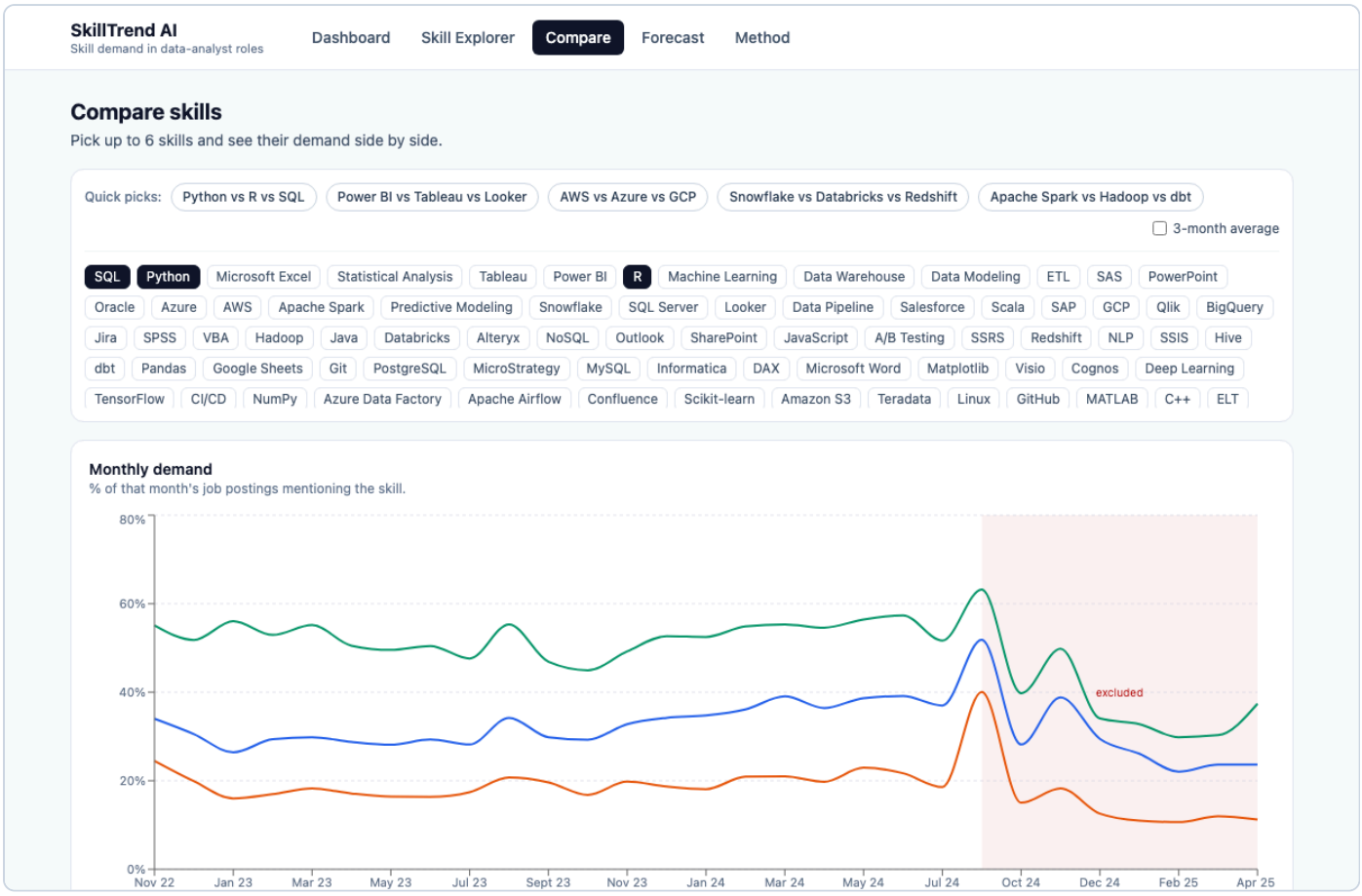
Skill Explorer



Every tracked skill with its demand, how many ads mentioned it, its direction, and its volatility. *Volatility* is how much a skill bounces around month to month — a steady trend is more trustworthy than a jumpy one.

Comparing skills

The question people actually ask is not "how popular is Tableau?" but "**should I learn Tableau or Power BI?**" — so skills can be put on one chart together.



Python vs R vs SQL across 29 months. One-click presets cover the common questions; any 6 skills can be picked manually.

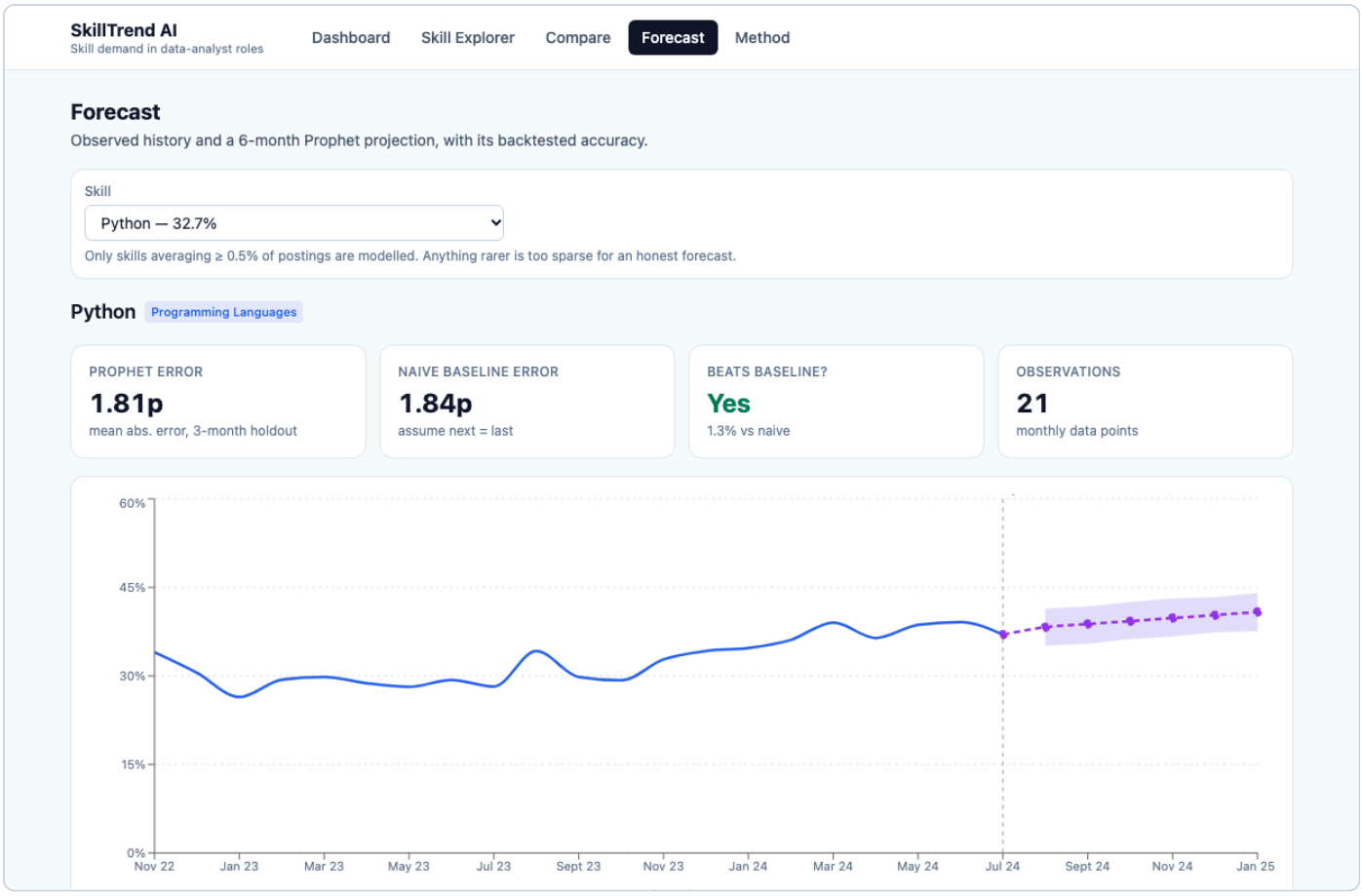
THE PINK REGION ON THE RIGHT

That is data the system **deliberately refuses to draw conclusions from**. Notice every line drops inside it — that is not the job market changing, it is a broken scraper. The full story is on page 9. It is drawn rather than hidden so you can see exactly what is being left out.

READING THE CHART HONESTLY

The y-axis is **% of that month's job ads**, not a count. That matters: the number of ads collected each month swings from 3,646 down to 272. With raw counts every skill would appear to collapse whenever the scraper slowed down. A percentage moves only when the *share* of employers asking for a skill moves.

The Forecast page



Python's demand history and its six-month projection.

Why this page looks the way it does

ELEMENT	WHY
Solid blue line	Observed history. This is *measured data* — it actually happened.
Dashed purple line	The model's projection. Dashed on purpose: it is a guess, and should never look like the same kind of thing as the blue line.
Shaded purple band	The 80% uncertainty interval. The model thinks the true value will land inside this band 8 times out of 10. A wide band means "I don't really know".
The four cards	How wrong the model was when tested — and how wrong the obvious guess would have been.

THE MOST IMPORTANT IDEA IN THIS PROJECT

A forecast means nothing on its own. You have to ask: **is it better than the obvious guess?** The obvious guess here is "*next month will look like last month*" — a **naive baseline**. On short, noisy data it is surprisingly hard to beat.

So every skill is **backtested**: hide the last 3 months, train on the rest, see which got closer. Both scores appear on the page. If the model loses, the page says so.

WHAT THE TEST ACTUALLY SHOWED

Prophet beat the naive baseline on **47 of 88 skills (53%)** — average error 0.783 vs 0.811 percentage points. In plain words: **a sophisticated model is only slightly better than assuming nothing changes.** For SQL it actually *lost*, and the SQL forecast page says exactly that.

The data: where it comes from

Source	*Data Analyst Job Postings* by Luke Barousse, on Kaggle (free)
How it was made	A scraper collected Google job-search results **every day** from November 2022 onward
Size	276 MB CSV · 61,953 job ads · 27 columns
Time span	Nov 2022 → Apr 2025 (29 months)
Job type	100% data-analyst roles
Download	One command through the free Kaggle API

WHY THIS DATASET AND NOT A BIGGER ONE

A broader dataset of all tech jobs (LinkedIn, 124,000 ads) was considered and rejected. It covered only 6–9 months.

Forecasting needs history. Predicting 6 months ahead from 6 months of data is not forecasting, it is guessing with a nice-looking chart. This dataset has **29 months**, so the trade was: fewer kinds of job, far more history.

The cost is honest and stated everywhere — React, Kubernetes and other web/DevOps skills barely appear, because data analysts are not asked for them.

What is in each job ad

The dataset has 27 columns. Only **9** were kept:

Kept ✓

`job_id`	unique id — used to spot duplicates
`date_time`	when the ad was seen
`description`	**the job text — the heart of it**
`title`	job title
`company_name`	employer
`location`	where
`via`	which job board
`schedule_type`	full-time, contract...
`description_tokens`	the dataset's own skill list — used only to *check our work*

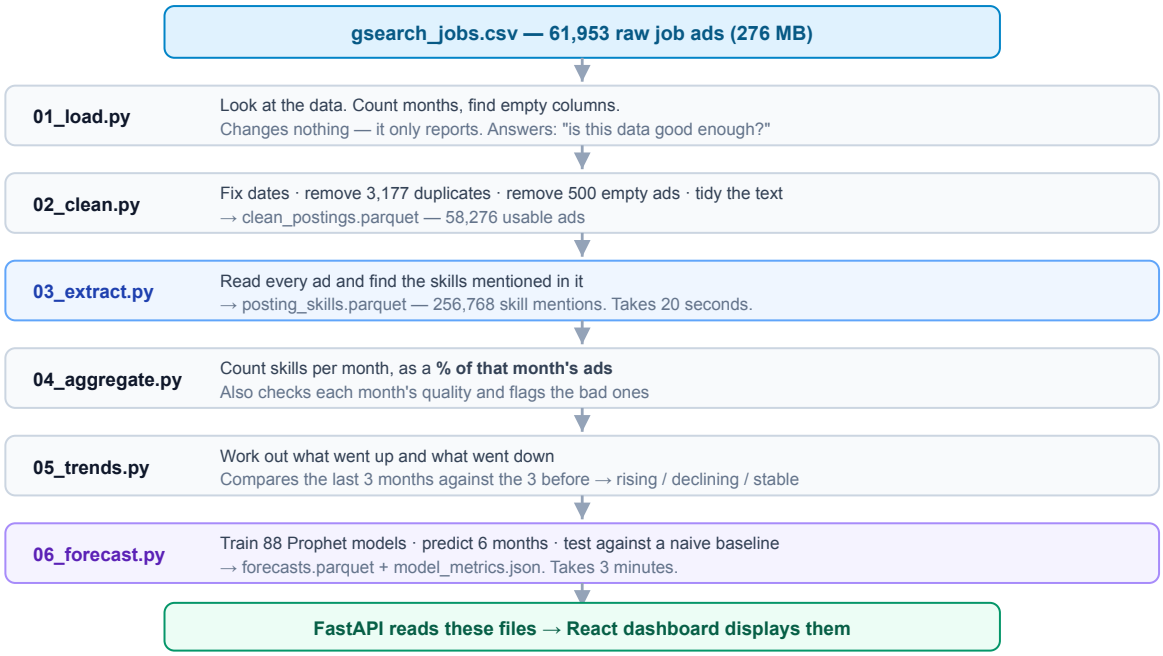
Dropped ✗

10 salary columns	84–93% empty
`commute_time`	100% empty
`posted_at`	says "15 hours ago" — useless
`search_term`	always "data analyst"
`thumbnail`, `extensions`	not relevant
`Unnamed: 0`, `index`	leftover row numbers

A column that is mostly empty cannot support any conclusion. Dropping them is not tidiness — it is correctness.

How it is made: the pipeline

Six Python scripts run one after another. Each takes a file, does one job, and writes a new file. Nothing is done twice and every step is checkable.



WHY SPLIT IT INTO SIX SCRIPTS INSTEAD OF ONE PROGRAM?

Because each step is slow and each step can be wrong. If cleaning takes a minute and forecasting takes three, you do not want to redo cleaning every time you tweak the model. Separate scripts mean you re-run only what changed — and each one prints a report, so a mistake is caught at the step that caused it.

WHY THE WEBSITE NEVER DOES THE HEAVY WORK

Training 88 forecasting models takes 3 minutes. If that happened every time someone opened the page, the site would be unusable. So the scripts run **once, beforehand**, and save their answers to files. The website just reads those files — which takes milliseconds.

This split (**train offline, serve online**) is how real machine-learning products are built.

The clever part: finding skills in text

A job ad is a paragraph of English. The computer has no idea that "Power BI" is a skill and "the ideal candidate" is not. Teaching it is the core technical work of this project.

How it works

A hand-built **dictionary** of 120 skills in 9 categories, stored in a plain text file (`skills/skills.yaml`). Each skill lists its alternative spellings:

```
Power BI:  [powerbi, power-bi]      # 3 spellings → 1 skill
PostgreSQL: [postgres, psql]
Go:         [golang, go programming] # NOT bare "go" — see below
```

The computer then scans each ad for those phrases, using a tool called **spaCy PhraseMatcher**. Whatever spelling it finds, it records the *canonical* name — so `powerbi` , `power-bi` and `Power BI` all count as one skill, not three.

Three traps, and how each is handled

THE TRAP	WHAT GOES WRONG	THE FIX
Ordinary words	"*go* above and beyond" is counted as the Go programming language	Bare "go" is banned. Only `golang` and `go programming` count.
Verbs	"*excel* at communication" is counted as Microsoft Excel	A match is rejected if the next word is "at", "in" or "as"
Words inside words	"no-sql" contains "sql", so both NoSQL *and* SQL get counted	Longest match wins — NoSQL is found, the stray SQL is dropped

DOES IT WORK? YES — MEASURABLY

The Kaggle dataset ships with its own skill list, so the two can be compared directly:

	THEIR EXTRACTOR	THIS PROJECT
Ads where a skill was found	78.6%	**82.2%**
Size of vocabulary	134 terms	**120 terms**
Reports "Go" more common than Java	Yes (wrong)	**No**
Can detect AI skills (ChatGPT, LLM...)	No — none in its list	**Yes**

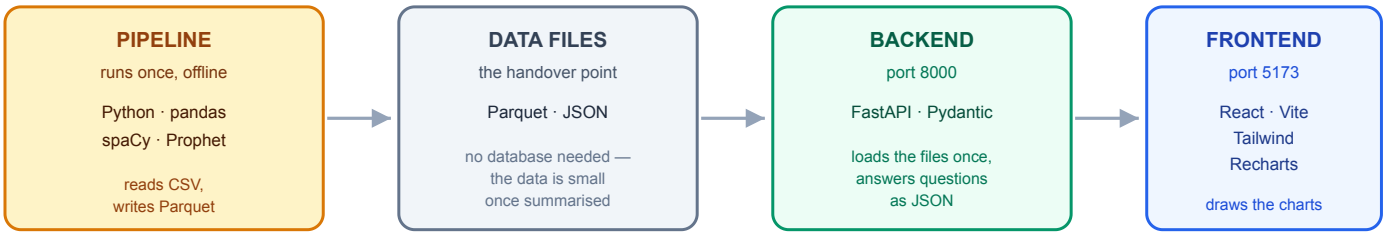
Better coverage from a *smaller* vocabulary, because the traps above are handled.

Why not use ChatGPT to find the skills?

	DICTIONARY (CHOSEN)	AN LLM
Same answer every time?	Yes	No
Can you explain a result?	Yes — point at the rule	Not really
Cost for 58,000 ads	Free	Real money
Speed	20 seconds	Hours

For counting things, being **repeatable and explainable** matters more than being clever.

Technology: what, where and why



TOOL	USED FOR	WHY THIS ONE
PIPELINE — Python		
pandas	Handling tables of data	The standard tool. Think Excel, but driven by code and able to handle millions of rows.
spaCy	Reading the job-ad text	Splits text into words correctly. Crucially it treats `C++` and `CI/CD` the same way in both the dictionary and the ad, so they match. Plain pattern-matching breaks on those.
Prophet	Forecasting	Made by Meta for exactly this: monthly business data with trends. Gives an uncertainty range, not just a single number, and is simple enough to explain.
Parquet	Saving results	A file format like CSV but smaller and much faster, and it remembers whether a column holds dates or text.
BACKEND — the server		
FastAPI	Serving the data	Turns Python functions into web addresses the website can call. Writes its own documentation page automatically.
Pydantic	Checking the data	Defines exactly what each response must contain, so a bug fails loudly on the server instead of quietly breaking the website.
Uvicorn	Running the server	The engine that actually listens on port 8000.
FRONTEND — the website		
React	Building the pages	Splits the UI into reusable pieces and redraws automatically when data changes.
Vite	Running & building	Updates the browser instantly when a file is saved.
Tailwind CSS	Styling	Styles applied directly in the markup — faster than maintaining separate CSS files for a dashboard with many small parts.
Recharts	The charts	Charts written as React pieces. Its shaded-area support is what makes the forecast uncertainty band straightforward.

COST: ZERO

Every tool above is free and open source. The dataset is free. Everything runs on one laptop. No cloud bill, no API keys, nothing to cancel.

Two things that nearly went wrong

These are the most valuable parts of the project, because both are mistakes that would have produced confident, completely wrong results.

PROBLEM 1

A broken scraper that looked like a market crash

The first trend report said: **29 skills declining, only 2 rising.** Python down 22%, SQL down 14%, Tableau down 18%. It looked like a dramatic finding about a collapsing job market.

It was completely false. Checking the underlying text revealed this:

PERIOD	TYPICAL AD LENGTH	ADS WHERE A SKILL WAS FOUND	SKILLS PER AD
Nov 2022 – Jul 2024	~3,000 characters	80–89%	5.4
Jan – Apr 2025	**~1,600 characters**	**65–68%**	**4.6**

From late 2024 the scraper started saving **only half of each job ad**. Less text means fewer skills found, so *every single skill's percentage fell at once*. The numbers were measuring the scraper, not the job market.

THE FIX

An automatic **quality gate** now checks every month before it is used. A month is rejected if it has fewer than 1,000 ads, or if its ads are shorter than 2,000 characters on average. **8 of 29 months were rejected**, leaving 21 good months (Nov 2022 – Jul 2024).

After the fix the picture is balanced and believable: **11 rising, 9 declining, 85 stable** — modern cloud data tools up, older BI tools down.

The rejected months are **still drawn on the Compare page**, shaded pink. They are excluded from conclusions, not from view.

PROBLEM 2

A forecast nobody had checked

It is easy to train a forecasting model, draw a smooth line into the future, and call it machine learning. The question nobody asks is: **is it better than a guess?**

So the model is tested against the simplest possible rule — "*next month will be the same as last month*":

0.783	0.811	53%
PROPHET ERROR	NAIVE GUESS ERROR	SKILLS WHERE PROPHET WON

WHAT THAT ACTUALLY MEANS

Prophet is better — by about **3%**. It wins on roughly half the skills and loses on the other half.

That is the honest answer when you have only 21 monthly data points. The project reports it on the dashboard and on every individual forecast page, including the ones where the model lost. The forecasts are useful as a *direction*, not as a number.

Running it, and where everything lives

Step 1 — set up

```
git clone <repo> && cd SkillTrend-AI
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

Step 2 — get the data

The data folder is not stored in Git — the raw file is 276 MB and every processed file can be regenerated. Put a free Kaggle API token at `~/.kaggle/kaggle.json`, then:

```
kaggle datasets download -d lukebarousse/data-analyst-job-postings-google-search   -f gsearch_jobs.csv -
p data/raw --unzip
```

Step 3 — run the pipeline

```
python scripts/01_load.py       # inspect only
python scripts/02_clean.py      # ~1 min
python scripts/03_extract.py    # ~20 sec
python scripts/04_aggregate.py  &&  python scripts/05_trends.py
python scripts/06_forecast.py   # ~3 min, 88 models
```

Step 4 — start the app (two terminals)

```
# terminal 1 — the server
cd backend && uvicorn app.main:app --reload

# terminal 2 — the website
cd frontend && npm install && npm run dev
```

Dashboard: `http://localhost:5173` · API docs: `http://127.0.0.1:8000/docs`

Where the code lives

```
backend/app/   main.py (server only) · core/config.py (all paths)
               services/ (loads data once) · schemas/ (response shapes)
               api/routes/ (the 8 web addresses)
frontend/src/  services/api.js (all backend calls) · hooks/ · components/
               pages/ (the 5 pages)
scripts/       01_load → 06_forecast, plus the extractor + its tests
skills/        skills.yaml — the 120-skill dictionary
data/          raw · processed · forecasts  (not stored in Git)
```

The 8 API addresses

`GET /api/health`	is the server alive?
`GET /api/summary`	everything the dashboard needs, in one call
`GET /api/categories`	the 9 skill categories
`GET /api/skills`	searchable list of all skills
`GET /api/skills/{name}`	one skill's summary
`GET /api/trends?skills=Python,SQL`	monthly history, up to 8 skills
`GET /api/trends/top`	biggest risers or fallers
`GET /api/forecast/{name}`	history + forecast + accuracy scores

What it cannot do

Every one of these is stated on the dashboard too. A project that says plainly what it does not know is worth more than one that looks impressive and cannot be defended.

LIMITATION	WHAT IT MEANS IN PRACTICE
Only data-analyst jobs	React, Kubernetes and most web/DevOps skills barely appear — not because they are unwanted generally, but because data analysts are not asked for them. This is not a picture of the whole tech industry.
Only one source	Everything came from Google job-search results. Any job not listed there is invisible.
Collection date, not posting date	The date is when the scraper saw the ad, usually within a day of publication. Fine for monthly figures, not exact.
A dictionary only finds what it lists	Any skill missing from the 120 reads as 0% — indistinguishable from a skill nobody wants.
A mention is not a requirement	"Python a plus" and "expert Python essential" are counted identically.
Forecasts are weak	21 months is very little history. Treat them as a direction, not a prediction.

Words worth knowing

Pipeline	A chain of steps where each one feeds the next
Parquet	A compact, fast file format for tables
API	A menu of requests one program can make to another
Endpoint	One item on that menu, e.g. `/api/summary`
Deduplicate	Remove repeated records so nothing is counted twice
Canonical name	The one official spelling a skill is recorded under
Alias	An alternative spelling that maps to the canonical name
Backtest	Hide recent data, predict it, and check how close you got
Baseline	A deliberately simple method used to check whether a clever one is actually better
MAE	Mean absolute error — on average, how far off the prediction was
Uncertainty interval	The range the model expects the real value to fall in
Volatility	How much a number jumps around. High volatility = less trustworthy trend

THE PRINCIPLE BEHIND THE WHOLE PROJECT

Keep three things separate and never blur them:

1. **What was observed** — counts taken from real job ads.
2. **What was calculated** — trends worked out from those counts.
3. **What was predicted** — a model's guess about months that have not happened yet.

On the dashboard these deliberately look different: solid lines for real history, dashed lines with a shaded band for forecasts, and pink shading over data that is not trustworthy enough to use.